A report submitted to

SDJ INTERNATIONAL COLLEGE - VESU

Affiliated To: Veer Narmad South Gujarat University, Surat

For, Partial requirement for the fulfillment of the degree of

Bachelor of Computer Applications (BCA)

BCA Sem VI

A.Y. 2025-26

Project Report On GlobeTrails

by

Roll No.	Seat No.	Name Of Student
310	9884	Rana Mehul Chandrakantbhai.
037	6713	Agarwal Samarth Rajkumar.
372	7271	Tivari Sani Pappubhai

Project Guide by:

Disha Nalwala

Acknowledgement

The success and final outcome of this project required a lot of guidance and assistance from many people and we are extremely fortunate to have got this all along the completion of our project work. Whatever we have done is only due to such guidance and assistance.

we would not forget to thank Principal Dr. Aditi Bhatt, IQAC coordinator and trust representative Dr. Vaibhav Desai, In-charge of PG Courses Dr. Vimal Vaiwala , In-charge of UG and Head of the UG Department Dr. Nainesh Gathiyawala and Project guide Disha Nalwala and all other Assistant professors of SDJ International College, who took keen interest on our project work and guided us all along, till the completion of our project work by providing all the necessary information for developing a good system.

We are extremely grateful to her/him for providing such a nice support and guidance though she/he had busy schedule managing the college dealings.
We are thankful and fortunate enough to get support and guidance from all Teaching staffs of IT Department which helped us in successfully completing our project work. Also, we would like to extend our sincere regards to all the non-teaching staff of IT Department for their timely support.

Rana Mehul Chandrakantbhai(9884)
Agarwal Samarth Rajkumar. (6713)
Tivari Sani Pappubhai. (7271)

I N D E X

Sr. No	Description	Page No.
1	Introduction	4
	1.1 Project description	4
	1.2 Project Profile	7
2	Environment Description	11
	2.1 Hardware and Software Requirements	11
	2.2 Technologies Used	11
3	System Analysis and Planning	14
	3.1 Existing System and its Drawbacks	14
	3.2 Feasibility Study	14
	3.3 Requirement Gathering and Analysis	16
4	Proposed System	22
	4.1 Scope	22
	4.2 Project Modules Functionalities	23
5	Detail Planning	34
	5.1 Data Flow Diagram / UML	34
	5.2 Data Dictionary	37
	5.3 Process Specification / Activity Flow Diagram	39
	5.4 Entity-Relationship Diagram / Class Diagram	41
6	System Design	43
	6.1 Database Design	43
	6.2 Input Design	44
	6.3 Output Design	46
7	Software Testing	50
	7.1 Unit Testing	50
	7.2 Integration Testing	51
	7.3 System Testing	52
8	Future Scope of Enhancements	53
9	Bibliography & Reference	55

1. Introduction

1.1 Project Description

GlobeTrails is a comprehensive, web-based Tour and Travel Management System designed to bridge the gap between travel enthusiasts and curated travel experiences. In the modern tourism industry, travelers seek seamless platforms to discover, plan, and book their journeys with trust and transparency. GlobeTrails addresses this need by providing a user-centric interface for browsing high-quality tour packages while offering a robust administrative backend for managing content, bookings, and user interactions.

The system is built as a Single Page Application (SPA) using the MERN stack (MongoDB, Express.js, React.js, Node.js), ensuring a responsive and dynamic user experience. Core functionalities include secure user authentication (including Google OAuth), a detailed tour catalog with category-based filtering, an integrated booking inquiry system, and a comprehensive Admin Panel for data management.

Target audience:

- Travelers: Individuals looking for organized tour packages for adventure, heritage, wildlife, etc.
- Administrators: Travel agency managers needing a digital dashboard to track inventory and inquiries.

1.1.1 Motivation and Background

The global travel and tourism industry is one of the fastest-growing economic sectors worldwide, contributing trillions of dollars annually to the global GDP. Despite this scale, a significant proportion of small to medium-sized tour operators continue to manage their operations through outdated, manual, or fragmented digital methods. Brochures, phone calls, spreadsheets, and static HTML websites remain the norm for many regional travel businesses, creating a wide and exploitable gap between traveler expectations and the actual booking experience available to them.

The modern traveler, shaped by the seamless convenience of platforms like Amazon, Uber, and Airbnb, now enters the travel planning process with fundamentally different

expectations. They expect to browse options at any hour, compare packages side by side, understand pricing transparently without calling an agent, and receive immediate confirmation of their inquiries without waiting days for a callback. When a travel agency fails to meet these expectations digitally, the traveler simply moves on to a competitor that does.

The motivation behind GlobeTrails was born directly from this observed market gap. The project was conceptualized to demonstrate that even a small-scale tour operator, without the resources of a MakeMyTrip or ThomasCook, can deploy a professional-grade, fully functional, and secure digital booking platform using entirely open-source, zero-licensing-cost technologies available through the Node Package Manager ecosystem.

Beyond the commercial motivation, the project carries significant educational purpose. Building GlobeTrails required applying a comprehensive range of computer science disciplines simultaneously — database schema design, RESTful API architecture, frontend state management, cryptographic security, cloud media storage integration, and responsive UI/UX design — making it an ideal capstone demonstration of full-stack software engineering competence at the undergraduate level.

1.1.2 Problem Statement

The travel and tourism sector faces a persistent, systemic inefficiency driven by the continued reliance on non-digital or partially-digital operational workflows. This creates measurable pain points for both the service provider (the travel agency or tour operator) and the service consumer (the traveler).

From the traveler's perspective, the core problem manifests as a severe lack of information accessibility and booking transparency. When tour details are only available through printed brochures, telephonic inquiry, or poorly maintained static websites, the traveler is forced into a high-friction discovery process that consumes significant time and effort before any actual booking can occur. Furthermore, without a structured booking system, travelers have no reliable mechanism to track the status of their inquiry, leaving them in a state of uncertainty after expressing interest.

From the tour operator's perspective, managing bookings through manual channels — WhatsApp messages, email threads, physical ledgers — introduces enormous administrative overhead and a high risk of data loss, double-booking, or missed inquiries.

There is no centralized system providing real-time visibility into how many bookings are pending, which tours are most popular, or what the current revenue trajectory looks like. Business intelligence, which is essential for operational planning, remains inaccessible in the absence of a structured data layer.

GlobeTrails was specifically designed to simultaneously solve both sides of this problem: providing travelers with a structured, transparent, self-service discovery and booking interface while providing operators with a data-driven administrative control panel that centralizes all business operations into a single, real-time digital dashboard.

1.2 Project Profile

1.2.1 Application Overview

GlobeTrails is a comprehensive web-based tour and travel management system designed to modernize how travel bookings are conducted. The platform serves as a digital intermediary between travel service providers and customers, addressing common industry pain points like fragmented information and manual booking processes. Developed as a Single Page Application (SPA) using React.js, the system provides a seamless, app-like user experience where all necessary functions—from browsing tours to managing bookings—occur within a single, dynamic interface without full page reloads.

The application represents a practical solution for small to medium-sized tour operators looking to establish an online presence with full booking capabilities. For travelers, it offers a centralized platform to discover, compare, and reserve tours across various categories including adventure, heritage, and wildlife experiences. The project's architecture emphasizes both user-friendliness and administrative control, creating a balanced ecosystem that serves both ends of the travel transaction process.

1.2.2 Architectural Design

The system is built on the MERN stack (MongoDB, Express.js, React.js, Node.js), selected for its consistency across development layers and strong community support. This JavaScript-based stack allowed for efficient development with a unified language approach, reducing context switching and simplifying both frontend and backend implementation.

Frontend Layer: Constructed with React.js using functional components and hooks, the interface implements a component-based architecture that promotes reusability and maintainability. State management is handled through React's built-in hooks for local state and Redux Toolkit for global application state, particularly for user authentication and session persistence. The use of React Router enables smooth navigation between views while maintaining the SPA experience.

Backend Layer: Powered by Node.js with the Express.js framework, this layer provides a RESTful API that handles all business logic, authentication, and data processing. The backend follows a modular structure with separate routes, controllers, and middleware functions, ensuring clean separation of concerns. Key middleware includes authentication verification, error handling, and request validation.

Database Layer: MongoDB serves as the primary data store, chosen for its flexibility with JSON-like documents that align naturally with JavaScript objects. The Mongoose ODM library provides schema validation, relationship management, and query building capabilities. The database design utilizes both embedded documents for closely related data (like tour itineraries) and document references for transactional relationships (like bookings connecting users and tours).

1.2.3 Data Management Approach

User Data: User information is stored with security as a priority. Passwords are hashed using BCryptJS before storage, and authentication relies on JSON Web Tokens (JWT) for stateless session management. The user schema distinguishes between regular users and administrators through role-based permissions.

Tour Content: Tours are modeled as comprehensive documents containing all relevant details—descriptions, pricing, duration, categories, and detailed itineraries. The itinerary structure uses embedded sub-documents for each day's activities, allowing complete tour information to be retrieved in a single database query. This design choice significantly improves performance for the most common user action: browsing tour details.

Media Assets: Tour images are managed through Supabase's cloud storage service. When administrators upload photographs, files are transmitted to Supabase via secure API calls, and only the resulting URLs are stored in the database(mongoDB). This approach offloads media storage and delivery responsibilities to a dedicated service while ensuring fast, optimized image delivery through their global CDN.

Booking Transactions: Booking Transactions: Bookings create associations between users and tours, capturing essential details like travel dates, participant counts, payment status, and current inquiry status. The booking schema supports a comprehensive lifecycle (pending → confirmed/rejected → cancelled) with integrated Razorpay payment processing. When a user submits a booking, a 40% advance deposit is collected through Razorpay's

secure payment gateway. The system creates a Razorpay order on the backend, processes the payment through Razorpay's checkout modal on the frontend, and verifies the payment signature cryptographically before updating the booking status. The schema tracks deposit amounts, balance amounts, Razorpay order IDs, payment IDs, and refund amounts separately. Administrative confirmation is required before the user can proceed to pay the remaining 60% balance. If an admin rejects a booking, a full refund of the deposit is calculated and recorded. User-initiated cancellations follow a tiered refund policy based on the number of days remaining before the tour date: 100% refund for cancellations made 30+ days in advance, 50% for 15-29 days, 25% for 7-14 days, and 0% for cancellations within 7 days of the tour.

1.2.4 Technical Implementation Standards

Security Measures: The application implements multiple security layers including JWT-based authentication, password hashing with salt rounds, role-based access controls, and input sanitization across all forms. MongoDB queries use parameterized inputs through Mongoose to prevent injection attacks, while React's built-in XSS protection safeguards against client-side vulnerabilities.

Responsive Design: The interface is built with mobile-first responsive principles using CSS Flexbox and Grid layouts. Breakpoints are set at standard device widths (mobile: 320px, tablet: 768px, desktop: 1024px) with particular attention to touch targets and readable typography on smaller screens. The design ensures consistent functionality across all modern browsers and devices.

Maintainability and Scalability: The codebase follows consistent naming conventions, modular component structure, and comprehensive documentation. While initially designed for smaller operations, the architecture supports scaling through horizontal deployment options, database indexing strategies, and the stateless nature of the JWT authentication system. Future expansion could include additional microservices without disrupting core functionality.

Development Practices: The project was developed using Git for version control, with feature-based branching and regular commits. Agile methodologies influenced the development process, with iterative improvements based on continuous testing and feedback. Post-deployment monitoring considerations include error tracking and performance analytics for ongoing optimization.

1.2.5 Objectives of the System

The primary objectives explicitly targeted by the GlobeTrails system design and implementation are as follows:

1. To develop a fully responsive, cross-platform, browser-based web application that enables travelers to independently discover, explore, and submit booking inquiries for tour packages without requiring any direct human assistance from the agency.
2. To implement a robust, role-based user authentication and authorization system that clearly differentiates between general user access and administrative access, ensuring all sensitive operations are protected against unauthorized access.
3. To design and deploy a comprehensive tour catalog management system that allows administrators to create, modify, and remove tour packages — including multi-day itineraries, pricing, categories, and multimedia content — through an intuitive graphical interface.
4. To establish a structured, trackable booking lifecycle management system that captures traveler inquiries, maintains real-time status visibility for both travelers and administrators, and supports administrative confirmation or rejection workflows.
5. To integrate cloud-based media storage through Supabase for reliable, scalable, and high-performance delivery of tour imagery across all device types and network conditions.
6. To implement social authentication through Google OAuth 2.0 in addition to traditional email/password registration, reducing registration friction and increasing user onboarding conversion rates.
7. To provide the administrative team with a centralized analytics dashboard presenting real-time aggregate statistics on user counts, tour inventory, booking volumes, and revenue metrics.
8. To ensure that all data transmission between the client and server is secured through industry-standard cryptographic protocols including JSON Web Token-based stateless authentication and BCrypt password hashing.

2. Environment Description

2.1 Hardware and Software Requirements

The development and deployment of GlobeTrails require specific environmental configurations to ensure optimal performance and compatibility.

Server-Side Requirements:

- Processor: Minimum 2 vCPU 2.4 GHz (Intel Xeon or AMD EPYC equivalent).
- RAM: 4GB Minimum (8GB Recommended for handling high concurrency during Booking transactions).
- Storage: SSD-based storage for database read/write efficiency; cloudbased storage-Supabase for hosting high-resolution tour imagery.
- Operating System: Windows 11,10
- Browser: The application leverages modern ES6+ JavaScript features. Used browsers with pervasive supported Google Chrome (v88+) and Microsoft Edge.
- Device Independence: The Responsive Web Design (RWD) implementation ensures compatibility across desktops, tablets (768px+), and mobile devices (320px+).

2.2 Technologies Used

The technology stack was selected based on a comprehensive feasibility study prioritizing scalability, non-blocking I/O, and component reusability.

Frontend:

- React.js (v19): Utilized for its Virtual DOM implementation, ensuring high-performance UI rendering. Functional components and Hooks (useState, useEffect, useContext) manage local state and side effects.
- Redux Toolkit: Implements the Flux architecture for global state management, specifically for handling User Session and Authentication states across the application wireframe.
- Vite: Adoption of Vite over Webpack significantly reduces build times and enhances Hot Module Replacement (HMR) during development.

Backend:

- Node.js & Express.js (v5): Provides the RESTful API layer. Express's middleware architecture allows for modular handling of requests, including body parsing, CORS management, and error handling.
- Passport.js: Integrated for modular authentication strategies, specifically isolating the logic for Google OAuth 2.0.
- Razorpay Node.js SDK: Official server-side SDK used to create payment orders, verify payment signatures, and interact with Razorpay's API. The SDK abstracts the complexity of direct API calls and handles authentication using the Razorpay Key ID and Key Secret stored as environment variables.

Database:

- MongoDB: A NoSQL document store chosen for its flexible schema (BSON), allowing for the nesting of complex data structures like "Tour Itineraries" and "Daily Schedules" without complex joins.
- Mongoose: Acts as the Object Data Modeling (ODM) layer, providing schema validation, pre-save hooks (middleware), and query building abstraction.

Security:

- BCryptJS: Used for key derivation functions to salt and hash user passwords, defending against Rainbow Table attacks.
- JSON Web Tokens (JWT): Facilitates stateless session management, eliminating the server-side overhead of maintaining session stores (e.g., Redis) for this scale of application.

Payment Processing:

- Razorpay: Integrated as the payment gateway for processing secure online transactions. Razorpay provides a PCI DSS compliant checkout interface that handles card payments, UPI, net banking, and wallet-based transactions. The integration follows Razorpay's recommended server-to-server order creation pattern, where the backend creates payment orders using the Razorpay Node.js SDK and the frontend renders the Razorpay Checkout modal for user-facing payment collection.

Payment verification is performed server-side using HMAC-SHA256 signature validation to ensure transaction integrity.

- crypto (Node.js built-in): Used for generating HMAC-SHA256 digests during Razorpay payment signature verification, ensuring that payment confirmation callbacks originate from Razorpay's servers and have not been tampered with during transmission.

Development Tools and Environment Configuration

The following development tools and environment configurations were employed throughout the GlobeTrails project lifecycle:

- **Visual Studio Code**: Primary source code editor, configured with extensions including ESLint for real-time JavaScript linting, Prettier for automatic code formatting enforcement, and the Thunder Client extension for in-editor HTTP API testing.
- **Git and GitHub**: Distributed version control with a remote repository hosted on GitHub. The branching strategy used a main branch for stable code, a development branch for active feature work, and individual feature branches for isolated component development.
- **Postman**: Used during the backend API development phase for comprehensive endpoint testing, including automated test scripts for authentication flow verification and booking lifecycle state transitions.
- **MongoDB Compass**: GUI-based database management tool used during development for direct schema inspection, query prototyping, and data verification after API operations.
- **Node Version Manager (NVM)**: Used to maintain consistent Node.js runtime versions across development environments, ensuring dependency compatibility with the project's specified Node.js LTS version.
- **dotenv**: Environment variable management library ensuring all sensitive configuration parameters (database connection strings, JWT secrets, Supabase API keys) are stored outside the codebase in untracked .env files.
- **Razorpay Dashboard:** Web-based administrative interface provided by Razorpay for managing API keys, monitoring transaction logs, reviewing payment analytics, and configuring payment settings. The test mode dashboard was used extensively during development to verify order creation, simulate payment flows, and debug transaction failures.

3. System Analysis

3.1 Existing System and its Drawbacks

Many traditional travel agencies still rely on:

- Manual Booking: Phone calls or physical visits, leading to inefficiency and time loss.
- Fragmented Information: Tour details scattered across brochures or static websites without real-time updates.
- Lack of Centralization: Admin data (users, tours, analytics) stored in spreadsheets or disparate systems.

Drawbacks:

- High operational costs.
- Slow response time to customer inquiries.
- Difficulty in updating tour inventory (pricing, availability).

3.2 Feasibility Study

Technical Feasibility: The project relies on JavaScript across the full stack (Isomorphic JavaScript). This uniformity minimizes context switching for developers and allows code sharing (e.g., validation logic) between client and server. The availability of robust libraries like mongoose for database interactions and lucide-react for iconography confirms high technical viability.

Operational Feasibility: The User Interface (UI) design focuses on HCI (Human-Computer Interaction) principles like "Visibility of System Status" and "Match between System and Real World." Consequently, travel agents or admins migrating from legacy paper-based systems to GlobeTrails will encounter a minimal learning curve.

Economic Feasibility: The economic viability of GlobeTrails is exceptionally strong due to the deliberate selection of entirely open-source, free-to-use technologies across every layer of the stack. The total infrastructure cost for development and initial deployment approaches zero:

- MongoDB Atlas provides a free-tier cluster (512MB storage) that comfortably accommodates thousands of user accounts, tour documents, and booking records during the initial operational phase.
- Supabase provides 1GB of free object storage for tour imagery, sufficient for hundreds of high-resolution photograph uploads before any paid tier becomes necessary.
- Vercel provides free hosting for the React frontend with automatic CDN distribution, zero-configuration HTTPS, and continuous deployment on every Git push.
- Render provides a free tier for hosting the Node.js backend server, sufficient for handling moderate API request volumes during the launch phase.

This zero-cost deployment model is particularly significant for the target audience of small to medium tour operators who may have limited technology budgets. The platform can be launched, operated, and tested in the real market without any upfront financial commitment, with a clear and predictable scaling cost pathway available when operational volume grows beyond the free-tier limits.

Legal and Compliance Feasibility:

All third-party services and libraries integrated within GlobeTrails are licensed under permissive open-source licenses (MIT, Apache 2.0) that explicitly permit their use in academic and commercial projects without licensing fees or attribution requirements beyond those specified in each library's license file. Google OAuth 2.0 integration complies with Google's API Services User Data Policy. The platform does not collect or store any payment card information, eliminating PCI-DSS compliance requirements within the current project scope.

3.3 Requirement Gathering

Functional Requirements:

The following functional requirements were established as the definitive behavioral specifications that the GlobeTrails system must satisfy. Each requirement is assigned a unique identifier and includes a detailed description of the expected system behavior, the triggering condition, and the acceptance criteria used to verify its correct implementation.

FR-01 — Identity Management (User Registration & Authentication)

The system must allow any visitor to create a new personal account by providing a unique email address, a display username, and a password meeting minimum complexity requirements. Upon submission, the system must validate the email format through regex pattern matching, verify that the submitted email does not already exist in the users collection, apply BCrypt hashing with a minimum of 10 salt rounds to the raw password before any database write operation, and persist the new user document with a default role of 'user'. The system must additionally support account creation and login through Google OAuth 2.0 as an alternative to the email/password pathway, allowing users who authenticate via Google to have a corresponding user document automatically created or retrieved from the database upon first login. Upon successful authentication through either pathway, the system must generate and return a signed JSON Web Token encoding the userId and role claims, which the client must store and include in the Authorization header of all subsequent requests to protected endpoints. The system must maintain this authenticated session state persistently across page refreshes and browser restarts until the user explicitly logs out or the token reaches its defined expiration threshold.

Acceptance Criteria: A new user document is successfully created in the users collection with the password field containing a BCrypt hash string (never plaintext). A valid JWT is returned on both email/password and Google OAuth login. Attempting to register with a duplicate email returns an HTTP 409 Conflict response. A logged-in user's session persists correctly after a full browser page refresh.

FR-02 — Password Recovery and Email Verification

The system must provide a complete self-service account recovery mechanism for users who have forgotten their password. When a user submits their registered email address to the forgot-password endpoint, the system must generate a time-limited one-time password (OTP), store a hashed representation of the OTP against the user's account record, and dispatch an automated email to the submitted address containing the OTP via the NodeMailer service. Upon the user submitting the received OTP to the verify-otp endpoint within the validity window, the system must validate the OTP against the stored hash and permit the user to set a new password, which must again be BCrypt-hashed before updating the user document. Expired or already-used OTPs must be rejected with an appropriate error response. The system must additionally support an email verification flow for newly registered accounts, setting the isVerified flag to true in the user document upon successful OTP validation.

Acceptance Criteria: Recovery email is received at the registered address within a reasonable delivery window. A valid OTP submitted within its validity period successfully permits password reset. An expired or incorrect OTP is rejected with HTTP 400 Bad Request. The isVerified flag transitions from false to true upon successful OTP verification.

FR-03 — Tour Catalog Browsing and Filtering

The system must present the complete collection of available tour packages to all visitors — both authenticated and unauthenticated — through a visually organized catalog interface. Each tour must be represented as a card component displaying at minimum the tour title, primary image, price per person, category tag, and duration. The catalog must support multi-attribute filtering enabling visitors to narrow the displayed results by tour category (from the predefined enum set including Adventure, Heritage, Wildlife, Beach, and Cultural), price range (through a minimum and maximum price boundary input), and trip duration (measured in days). Filters must be applicable in combination — a user must be able to simultaneously filter for Beach tours under a specified price ceiling of a given duration. The system must additionally support navigation to a dedicated tour detail page for any catalog entry, where the complete tour information is displayed including the full multi-day itinerary with per-day activity descriptions, the complete image gallery, all applicable amenities, pricing breakdown, and a booking inquiry form.

Acceptance Criteria: All tours present in the database are rendered in the catalog on page load. Applying a category filter displays only tours whose category field matches the selected value. Applying a price range filter excludes tours whose pricePerPerson falls outside the specified bounds. Navigating to a tour detail page displays all embedded itinerary data for that specific tour document without requiring additional page loads.

FR-04 — Booking Inquiry Submission and Tracking

The system must allow any authenticated user to submit a booking inquiry for any tour package displayed in the catalog. The booking inquiry form must capture the user's preferred travel start date — with the date picker enforcing a minimum selectable date equal to the current calendar date to prevent past-date submissions — and the total number of guests in the traveling party. Upon form submission, the system must create a new Booking document in the database that references both the submitting user's ObjectId and the target tour's ObjectId, sets the inquiryStatus field to its default value of 'pending', and calculates the finalPrice as the product of the tour's pricePerPerson and the submitted guest count. The system must immediately dispatch a confirmation email to the user's registered email address acknowledging receipt of the inquiry. The system must provide authenticated users with a dedicated "My Bookings" page listing all their submitted booking inquiries with the current status of each — pending, confirmed, or rejected — along with the tour name, travel date, guest count, and total price, enabling real-time tracking of the inquiry lifecycle without requiring direct contact with the agency.

Payment Processing and Deposit Collection:

The system must integrate a secure online payment gateway (Razorpay) to collect a mandatory 40% advance deposit from the user at the time of booking submission, before the booking is registered as a valid inquiry. The payment flow must follow a server-validated order creation pattern: the backend must create a Razorpay order for the calculated deposit amount, the frontend must render the Razorpay Checkout modal initialized with the backend-generated order ID and amount, and upon successful payment, the backend must cryptographically verify the payment signature using HMAC-SHA256 before updating the booking's payment status to 'deposit_paid'. If the user dismisses the payment modal or the payment fails, the system must automatically delete the pending booking record from the database to prevent orphaned entries. The booking must remain in 'pending' inquiry status after deposit payment, awaiting administrative confirmation. Only after admin confirmation

should the user be presented with the option to pay the remaining 60% balance through a second Razorpay transaction.

Acceptance Criteria: A Razorpay order is successfully created on the backend with the correct deposit amount (40% of finalPrice) in paise. The Razorpay Checkout modal opens on the frontend displaying the correct payment amount. Successful payment updates the booking's paymentStatus to 'deposit_paid' and stores the Razorpay payment ID. Failed or cancelled payments result in the pending booking being permanently deleted from the database. The booking's inquiryStatus remains 'pending' after deposit payment, not 'confirmed'. The 'Pay Remaining Balance' button appears on the user's profile page only for bookings that have been administratively confirmed.

FR-05 — Administrative Tour Inventory Management

The system must provide authenticated administrator users with complete Create, Read, Update, and Delete (CRUD) authority over the entire tour package catalog through the Admin Panel interface. The tour creation form must support all tour metadata fields including title, description, category (constrained to the predefined enum values), price per person, duration in days, a dynamically expandable multi-day itinerary builder allowing administrators to add and remove individual day entries, and a multi-image uploader that transmits selected files to the Supabase cloud storage service and embeds the resulting public CDN URLs in the tour document's images array. The system must support editing any field of an existing tour through a pre-populated edit form that submits a PUT request to update the document in the database. The system must support permanent deletion of any tour document from the catalog through a confirmed DELETE operation. All create, update, and delete operations must be accessible exclusively to users whose JWT contains role: 'admin'; any attempt by a standard user to invoke these endpoints must be rejected with HTTP 403 Forbidden before any database operation is executed.

Acceptance Criteria: A newly created tour appears immediately in the public catalog after successful admin submission. Image files uploaded during tour creation are retrievable via their Supabase CDN URLs. Editing a tour and saving updates correctly reflects the changed values in both the database document and the public-facing tour detail page. Deleting a tour removes it from the catalog and prevents further access to its detail page. A non-admin JWT is rejected by the verifyAdmin middleware with HTTP 403 before any tour modification logic executes.

FR-06 — Booking Lifecycle Management (Administrative)

The system must provide administrator users with a centralized booking management interface that displays all booking inquiries submitted across all registered users of the platform in a tabular format. Each booking record displayed must include the submitting user's name and email, the target tour name, the preferred travel date, the guest count, the calculated final price, and the current inquiry status. The system must allow administrators to transition any booking's inquiryStatus from 'pending' to either 'confirmed' or 'rejected' through direct action buttons within the booking management table, with each status change executed as an atomic PATCH request to the database that updates only the inquiryStatus field of the targeted document. Status changes must be immediately reflected in both the administrator's booking table and the corresponding user's My Bookings page upon the next data fetch, ensuring consistent state visibility across both user roles. The system must enforce that inquiryStatus can only be set to values within the defined enum set; any attempt to set an arbitrary status value through a direct API call must be rejected by Mongoose schema validation.

Additionally, when an administrator rejects a booking that has already received a deposit payment, the system must calculate a 100% refund of the deposited amount and record the refund amount in the booking document. The booking's payment status must be updated to 'refunded', and the refund amount must be visible to both the user on their profile and the administrator on the booking management table. For user-initiated cancellations, the system enforces a tiered refund policy: 100% refund for cancellations made 30 or more days before the tour date, 50% for 15-29 days, 25% for 7-14 days, and 0% refund for cancellations within 7 days of the tour.

Acceptance Criteria: The admin booking table displays all bookings from all users, not only the administrator's own bookings. Clicking the Confirm action on a pending booking updates its inquiryStatus to 'confirmed' in the database. The updated status is correctly reflected in the targeted user's My Bookings view. Attempting to set inquiryStatus to a value outside the defined enum through a direct API call returns a Mongoose validation error and HTTP 400 response.

FR-07 — Administrative Dashboard Analytics

The system must provide the administrative team with a real-time business intelligence dashboard as the default landing page of the Admin Panel. The dashboard must display aggregate statistical metrics computed by executing concurrent API requests to retrieve the

total count of registered user accounts, the total count of active tour packages in the catalog, the total count of all booking inquiries across all statuses, and the cumulative revenue calculated as the sum of all finalPrice values across all confirmed bookings. These metrics must be presented in clearly labeled summary cards that update to reflect the current database state on each dashboard page load. The dashboard must additionally display a recent bookings panel showing the latest incoming inquiries with sufficient detail for the administrator to identify the tour, user, and date without navigating to the full booking table, along with quick-action status update buttons enabling immediate processing of the pending inquiry queue directly from the dashboard home page.

Acceptance Criteria: All four aggregate metrics (users, tours, bookings, revenue) are displayed correctly on dashboard load, matching the actual counts present in the MongoDB collections. The cumulative revenue figure correctly sums only confirmed bookings' finalPrice values, excluding pending and rejected bookings. The recent bookings panel displays the most recently created booking inquiries in descending chronological order. Dashboard statistics update to reflect database changes upon the next page load or component remount.

4. Proposed System

4.1 Scope

The GlobeTrails system is designed to encompass the complete digital workflow of modern tour booking operations. The platform's scope extends across the entire user journey—from initial discovery and exploration to final booking confirmation and administrative management. This comprehensive approach ensures that both travelers and tour operators have access to all necessary tools within a unified ecosystem.

System Boundaries and Functional Coverage:

In-Scope Components:

Web Application Platform:

- A fully responsive, cross-browser compatible web interface accessible via standard web browsers
- Client-side application built with React.js providing dynamic, app-like user experience
- Mobile-optimized interface ensuring functionality across smartphones, tablets, and desktops

Core Database Infrastructure:

- MongoDB database for primary data storage including user profiles, tour information, and booking records
- Document-based schema design optimized for travel industry data structures
- Secure data access layers with proper authentication and authorization protocols
- API Server and Backend Services:
- RESTful API architecture built with Node.js and Express.js
- Comprehensive API endpoints covering user authentication, tour management, booking processing, and administrative functions
- Integration with third-party services for cloud storage (tour images) and authentication (Google OAuth)

Administrative Interface:

- Dedicated admin dashboard with role-based access control
- Content management system for tour creation, modification, and removal
- Booking management interface for status updates and customer communication
- Basic analytics and reporting functionalities for business insights
- Public tour catalog with advanced filtering and search capabilities
- Detailed tour presentation pages with comprehensive itineraries and multimedia content
- User registration and authentication system (email/password and social login)
- Booking request submission and status tracking

4.2 Project Modules

4.2.1 Authentication & Security Module

Overview: Authentication is the gatekeeper of the GlobeTrails system. Unlike monolithic applications that might rely on server-side sessions and cookies, GlobeTrails implements a stateless JWT (JSON Web Token) architecture. This ensures the backend remains scalable and decouples the frontend from specific server instances (crucial for eventual microservices architecture).

User Registration Workflow:

- ✓ Input Validation: When a user submits the signup form, the frontend performs initial regex validation on the email and password complexity checks.
- ✓ API Payload: A POST request is sent to /api/auth/register containing the sanitized payload.
- ✓ Uniqueness Check: The backend controller invokes User.findOne({ email }). Indexes on the email field in MongoDB allow this lookup to occur in O(log n) time.
- ✓ Cryptographic Hashing: Upon verification that the user does not exist, the password is passed through bcryptjs. We utilize a salt round of 10, balancing security against CPU processing time.
- ✓ Persistence: The new user document, containing the hashed password and default role: 'user', is written to the database.

Login Workflow & Session Management:

- ✓ Credential Verification: For email login, bcrypt.compare() compares the plaintext input against the stored hash. For Google login, passport-google-oauth20 exchanges the authorization code for an ID token and profile data.
- ✓ Token Generation: Upon successful verification, the server signs a JWT using the RS256 or HS256 algorithm and a private JWT_SECRET. The payload includes non-sensitive claims: userId and role.
- ✓ Storage Strategy: The token is sent to the client. The React application stores this token in localStorage or an HttpOnly Cookie. Within the React state (Redux), a boolean flag isAuthenticated is set to true.
- ✓ Role-Based Access Control (RBAC): RBAC is implemented via custom Express middleware.

Middleware Logic: A higher-order function verifyAdmin intercepts sensitive routes (e.g., POST /tours, DELETE /users/:id).
Token Decoding: The middleware decodes the JWT to extract the role claim.
Authorization: If req.user.role !== 'admin', the request is terminated immediately with a 403 Forbidden status code, preventing unauthorized logical access to administrative controllers
.

4.2.2 Tour Catalog & Content Management Module

Concept Explanation: The Tour Catalog is the core product engine of GlobeTrails. It transforms raw database entries into visually appealing, searchable travel packages. This module handles the complex data modeling required to represent multi-day itineraries, diverse amenities, and pricing structures.

Data Modeling & Design Decisions: The Tour schema in MongoDB is designed to be read-heavy. Since users browse tours far more often than admins update them, we utilize a denormalized schema structure.

Nested Itineraries: Instead of a separate SQL-style "Itinerary" table joined by ID, the itinerary is embedded directly within the Tour document as an array of objects [{ day: 1, activity: "..." }, { day: 2... }]. This allows the application to fetch the complete tour details in a single database query, significantly reducing latency.

Categorization: The schema enforces strict Enum validation for categories (e.g., "Wildlife Tour", "Beach Tour"). This guarantees data integrity and ensures filters on the frontend always return consistent results.

Technical Workflow (Frontend):

Fetching: The Tours.jsx component initiates an asynchronous axios.get() call on mount via the useEffect hook.

State Management: Data is stored in local component state tours[].

Rendering: The map() function iterates over the array to render TourCard components.

Dynamic Routing: clicking a "View Details" button pushes the user to /tours/:id. React Router extracts the :id param, allowing the TourDetails component to fetch specific data for that unique ID.

4.2.3 Booking & Reservation Engine Module

Workflow:

User Intent: On the TourDetails page, a logged-in user selects a preferredStartDate and enters the number of guests. The system calculates the total price (pricePerPerson × guests) and displays a payment breakdown showing the Total Tour Price, the 40% Advance Deposit amount, and the 60% Remaining Balance.

Razorpay Script Loading: Before proceeding, the frontend dynamically loads the Razorpay Checkout script (https://checkout.razorpay.com/v1/checkout.js) to ensure the payment modal can be rendered.

Data Submission: The form submission triggers a POST /bookings request with the payload containing the tour ID, preferred start date, guest count, contact details, and the total finalPrice. The Authorization header includes the JWT Bearer token.

Backend Processing:

The controller extracts userId from the decoded JWT token.

It calculates the deposit amount as Math.round(finalPrice × 0.4) and the balance amount as the remainder.

A new Booking document is created referencing both User and Tour, with inquiryStatus: 'pending' and paymentStatus: 'pending'.

A Razorpay order is created using the Razorpay Node.js SDK with the deposit amount converted to paise (multiplied by 100).
The Razorpay order ID is stored in the booking document.
The Razorpay order object and booking data are returned to the frontend.
Payment Modal: The frontend initializes the Razorpay Checkout modal using the order ID and amount received from the backend. The user completes the payment using cards, UPI (success@razorpay in test mode), or net banking.

Payment Verification: Upon successful payment, the Razorpay handler callback sends the razorpay_order_id, razorpay_payment_id, and razorpay_signature to the backend's /bookings/verify-payment endpoint. The backend reconstructs the expected signature using HMAC-SHA256 with the Razorpay Key Secret and compares it against the received signature. If verified, the booking's paymentStatus is updated to 'deposit_paid'.

Failure Handling: If the payment fails or the user dismisses the modal, the frontend calls DELETE /bookings/cancel-payment/:id to permanently remove the pending booking, preventing orphaned records.

Admin Review: The booking remains in inquiryStatus: 'pending' state. The admin reviews it on the Admin Panel and either confirms or rejects it.

Balance Payment: Once an admin confirms the booking, the user's profile page displays a "Pay Remaining Balance" button. Clicking it triggers a second Razorpay transaction for the remaining 60% amount, following the same order creation → checkout → verification flow.

Process Specification — Status Lifecycle:

A finite state machine governs both the booking inquiry status and the payment status:

Inquiry Status Lifecycle:

Pending: Initial state upon creation and successful deposit payment. Awaiting administrative review.
Confirmed: Admin has reviewed capacity and accepted the booking. User can now proceed to pay the remaining balance.
Rejected: Admin has declined the request. If deposit was paid, a 100% refund is calculated and recorded. Payment status changes to 'refunded'.

Cancelled: User has voluntarily cancelled. Tiered refund policy applies based on days remaining before tour date.
Payment Status Lifecycle:

Pending: Initial state. No payment received yet.
Deposit Paid: 40% advance deposit successfully processed through Razorpay.
Full Paid: Both deposit (40%) and remaining balance (60%) successfully processed.
Refunded: Refund calculated due to admin rejection or user cancellation.
This dual-status architecture is enforced at the schema level using Mongoose Enums for both inquiryStatus and paymentStatus fields.

4.2.4 Administrative Dashboard (Admin Panel)

Concept Explanation: The Admin Panel is a specialized, secured Single Page Application accessible only to users with the admin role. It serves as the command center for the entire GlobeTrails platform.

Key Features & Implementation:

Dashboard Analytics:
Implementation:
Dashboard.jsx component aggregates data. It runs parallel Promises (Promise.all) to fetch counts from users, tours, and bookings collections simultaneously.
Visualization: Data is presented in "Stat Cards" providing immediate business intelligence (e.g., "Total Revenue: ₹125,000").

Tour Inventory Management (CRUD):

- Create: A sophisticated multi-step form allows admins to input tour metadata. The TourUploadForm handles complex state for dynamic itinerary arrays (adding/removing days).
- Update: An edit mode pre-fills the form with existing data, performing a PUT request on submission.
- Delete: A safeguarded delete action removes tours from the public catalog.

Booking Management:
Admins view a tabular list of all bookings with a dedicated Payment Status column showing color-coded badges — grey for 'Pending', orange for 'Deposit Paid', green for 'Fully Paid',

and red for 'Refunded'. Each row also displays the deposit amount and remaining balance for quick financial review. Action buttons allow the admin to confirm or reject pending bookings. When an admin rejects a booking with a paid deposit, the system automatically calculates a 100% refund of the deposited amount, updates the payment status to 'refunded', and records the refund amount. This ensures complete financial traceability within the admin interface.

4.3 Module-wise Objectives, Functionalities & Constraints

Module 1: Authentication & Security Module

Objectives: To establish a completely secure, stateless identity management layer that controls access to all protected resources within the GlobeTrails platform, ensuring that each user's data remains exclusively accessible to them and that administrative capabilities remain entirely inaccessible to general users.

Functionalities:

- Validates all registration input on both the client side (regex format checks) and server side (Mongoose schema validation) before any database write is executed.
- Applies BCrypt salting and hashing with 10 salt rounds to all raw password strings before persistence, ensuring no plaintext credentials are ever written to the database.
- Integrates Google OAuth 2.0 via Passport.js, exchanging the Google authorization code for an ID token and profile object, then creating or updating the corresponding user document in MongoDB.
- Signs and issues JSON Web Tokens encoding the userId and role claims upon successful authentication through either channel.
- Decodes and validates JWT signatures on every protected route request through the isAuthenticated middleware, rejecting expired or malformed tokens with HTTP 401 responses.
- Enforces Role-Based Access Control through the verifyAdmin middleware, returning HTTP 403 Forbidden for any non-admin attempt to access administrative API routes.

Input: Email, password, username (registration); Email, password (login); Google OAuth authorization code (social login). **Output:** Signed JWT token, sanitized user object (password field excluded), HTTP status code and error message JSON on failure.

Constraints:

- Email field must be unique across the entire users collection; the MongoDB unique index on email enforces this at the database level, not just at the application level.
- JWT tokens carry a fixed expiration duration; after expiry, the user must re-authenticate and obtain a new token. No token refresh mechanism is implemented in the current version.
- Google OAuth requires active internet connectivity to reach Google's authorization servers; the feature is non-functional in completely offline environments.
- The RBAC system is binary (user vs. admin); granular permission levels within the admin role are not supported in the current implementation scope.

Module 2: Tour Catalog & Content Management Module

Objectives: To serve as the primary product discovery engine of the platform, presenting all available tour packages in a visually rich, filterable, and navigable interface while providing administrators with complete create, read, update, and delete authority over the tour catalog.

Functionalities:

- Fetches all tour documents from MongoDB via an authenticated GET endpoint and renders them as individual TourCard components within a responsive CSS Grid layout.
- Implements client-side filtering by category (Adventure, Heritage, Wildlife, Beach), price range, and duration without additional API calls, using in-memory array filtering on the locally cached tours state array.
- Renders a dedicated TourDetails page for each tour, displaying the complete multi-day itinerary, full image gallery, pricing breakdown, category tags, and duration information retrieved via a single parameterized GET request.
- Provides an admin-exclusive multi-step TourUploadForm supporting dynamic itinerary day addition and removal, image file selection with preview, and all tour metadata fields, submitting to the POST /tours endpoint on completion.
- Handles image uploads by transmitting selected files directly from the frontend to the Supabase Storage API, storing only the resulting public CDN URLs in the MongoDB tour document.
- Supports tour modification through a pre-populated edit form submitted to the PUT /tours/:id endpoint and tour removal through a confirmed DELETE /tours/:id request.

Input: (Public) No authentication required for browsing. (Admin) JWT Bearer token in Authorization header; tour metadata fields and image files for create/update operations.

Output: Array of tour documents for catalog view; single tour document with full embedded itinerary for detail view; confirmation response on create/update/delete.

Constraints:

- The category field is strictly validated against a predefined Mongoose enum array; any tour submitted with an unrecognized category string is rejected by schema validation at the database level.
- Image files must be uploaded to Supabase before the tour document is saved to MongoDB; if the Supabase upload fails, the tour creation process must be aborted to prevent database records without associated media.
- The client-side filtering mechanism operates on the complete tours dataset loaded on page mount; for very large catalogs exceeding several hundred tours, server-side pagination and filtering should be implemented to maintain performance.
- Tour deletion is permanent and irreversible within the current system; no soft-delete or archival mechanism is implemented, and deleted tour documents cannot be recovered without direct database intervention.

Module 3: Booking & Reservation Engine Module

Objectives: To capture authenticated user booking intent in a structured, database-persisted format, establish a trackable booking lifecycle from initial inquiry through final administrative confirmation or rejection, and provide real-time status visibility to both the traveler and the administrative team.

Functionalities:

- Renders a booking inquiry form on the TourDetails page accessible only to authenticated users, collecting preferred start date and guest count inputs with client-side validation constraints.
- Submits booking data via an authenticated POST /bookings request, with the backend controller extracting the userId directly from the verified JWT payload rather than trusting any user-supplied ID in the request body.
- Creates a new Booking document referencing both the User ObjectId and the Tour ObjectId, setting inquiryStatus to the default 'pending' value and calculating the finalPrice as pricePerPerson multiplied by the guest count.

- Retrieves and displays all bookings associated with the authenticated user on the My Bookings dashboard, showing tour name, travel date, guest count, total price, and current status for each entry.
- Provides the administrative panel with a complete tabular view of all bookings across all users, supporting status updates from 'pending' to 'confirmed' or 'rejected' via PATCH requests.
- Triggers a NodeMailer email notification to the booking user's registered email address upon successful booking submission confirming that the inquiry has been received.

Input: JWT Bearer token; tour ID from URL parameter; preferredStartDate (Date picker), guests (numeric input). **Output:** Created Booking document with generated ObjectId, inquiryStatus: 'pending', and calculated finalPrice; toast notification on frontend; confirmation email to user.

Constraints:

- The booking endpoint requires a valid, non-expired JWT token; unauthenticated users are redirected to the sign-in page before any booking form is displayed.
- Date validation enforces that the preferredStartDate cannot be set to any date in the past; the date picker component enforces a minimum selectable date equal to the current day.
- The Razorpay payment gateway requires active internet connectivity to reach Razorpay's API servers for order creation and payment verification; the booking flow cannot be completed in offline environments.
- The payment amount sent to Razorpay must be in the smallest currency subunit (paise for INR); the deposit amount is calculated as Math.round(finalPrice × 0.4) × 100 paise to avoid floating-point precision errors.
- The Razorpay Checkout script is loaded dynamically at runtime; if the script fails to load due to network issues or content blockers, the booking process cannot proceed and the user is notified to retry.
- Payment signature verification relies on the RAZORPAY_KEY_SECRET stored as a backend environment variable; if this secret is compromised or misconfigured, payment verification will fail for all transactions.

Module 4: Administrative Dashboard Module

Objectives: To provide the travel agency's operational team with a centralized, real-time command center that consolidates all business intelligence and content management capabilities into a single secured interface, enabling data-driven operational decisions without requiring direct database access.

Functionalities:

- Aggregates and displays real-time business metrics — total registered users, total active tour packages, total booking inquiries, and cumulative revenue — by executing parallel API calls through Promise.all() on dashboard component mount.
- Renders a recent bookings section on the dashboard home page, displaying the latest five incoming inquiries with booking details and quick-action status update buttons for immediate administrative response.
- Provides a complete user management table listing all registered accounts with registration date, email, and verification status, supporting account deletion for spam or invalid accounts.
- Hosts the complete tour inventory management interface enabling full CRUD operations on all tour packages through the TourUploadForm and inline edit and delete actions within the tour management table.
- Displays a comprehensive booking management table with all historical and pending bookings filterable by status and payment status, allowing administrators to efficiently process the pending inquiry queue.
- All administrative routes are protected at both the React Router level (redirect non-admin users before rendering) and the API level (verifyAdmin middleware rejects non-admin JWT claims), implementing defense-in-depth access control.

Input: Admin JWT Bearer token; administrative actions (status update clicks, tour form submissions, delete confirmations). **Output:** Aggregated statistics cards, tabular data grids with pagination, success/error toast notifications, real-time UI updates reflecting database state changes.

Constraints:

- The administrative dashboard is accessible exclusively to users whose JWT payload contains role: 'admin'; this role must be assigned directly in the database by an existing administrator, as there is no self-registration pathway to admin status.
- The parallel API call pattern using Promise.all() means that if any single statistics API endpoint fails, the entire dashboard statistics section displays an error state rather than partial data.
- The current analytics implementation provides aggregate totals only; time-series charts, trend analysis, and date-range filtered reporting are identified as future scope enhancements not present in the current version.

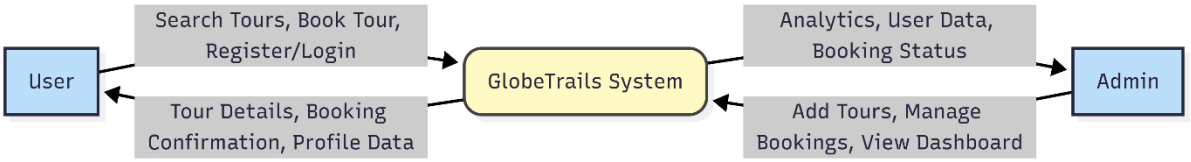
5. Detail Planning

5.1 Data Flow Diagram (DFD)

The Data Flow Diagrams (DFD) illustrate how data moves through the GlobeTrails system, transforming from user inputs to processed database records.

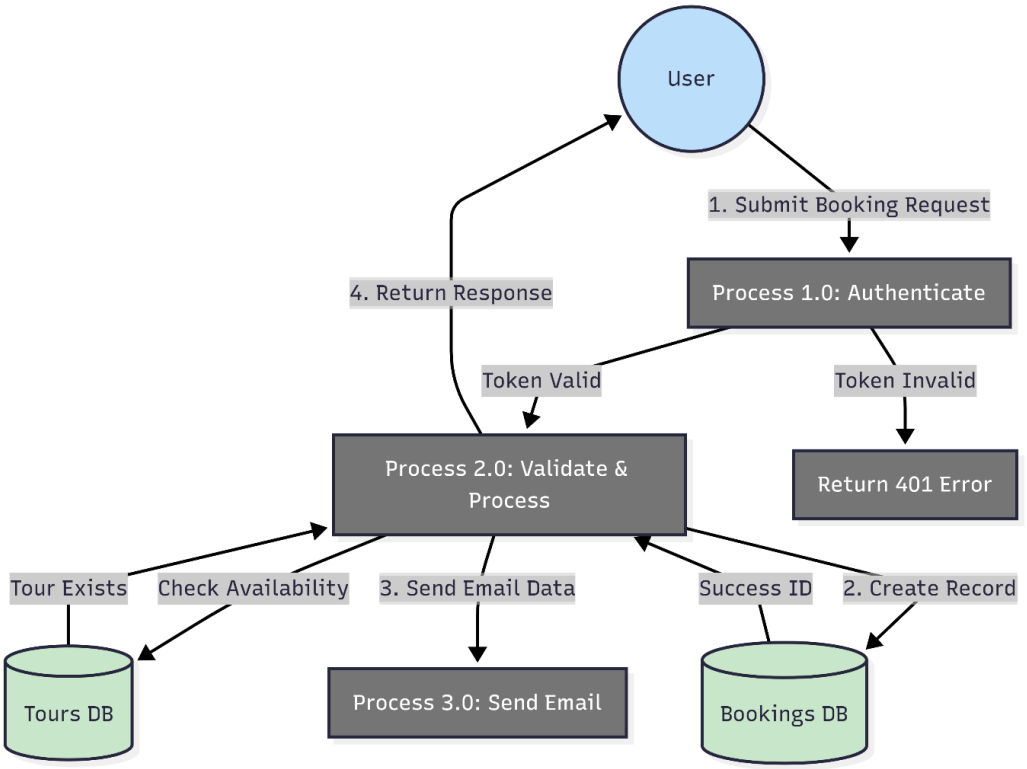
Level 0 Context Diagram:

This high-level overview shows the interaction between external entities (User, Admin) and the GlobeTrails System as a complete black box.



Level 1 DFD : Data Flow (booking process):

This diagram breaks down the "Booking Process" into specific sub-processes to show the internal flow of logic.



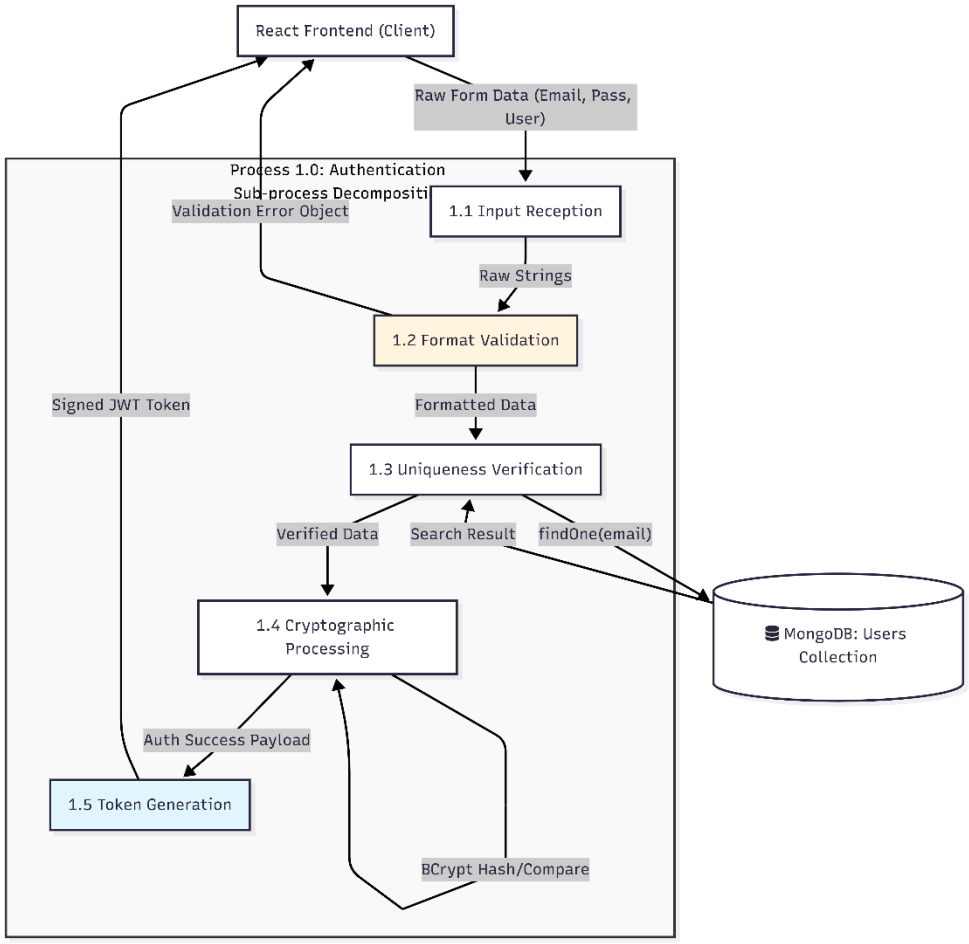
- **Node 1 - Booking Request:** The user initiates the flow by submitting a form on the Tour Details page.

- **Node 2 - Auth Validation:** The request is intercepted by the isAuthenticated middleware to verify the user's identity via JWT.
- **Node 3- Validation:** The system checks business logic (e.g., date validity, duplicate bookings).
- **Node 4 - Persistence:** The validated data is saved to the Bookings collection in MongoDB.
- **Node 5 - Notification:** The system triggers an email service to notify the user.

Level 2 DFD — Authentication Sub-process Decomposition

The Level 2 DFD for the Authentication sub-process decomposes the identity management flow into its specific internal processing nodes:

- **Node 1.1 — Input Reception**: Receives raw form data from the React frontend, including email string, password string, and username string for registration flows, or email and password for login flows.
- **Node 1.2 — Format Validation**: Applies regex pattern matching to verify email format conformity and password minimum length compliance. Returns a validation error object to the client without any database interaction if validation fails, conserving database query resources.
- **Node 1.3 — Uniqueness Verification**: Executes a findOne database query against the users collection's indexed email field to determine whether the submitted email address already exists in the system.
- **Node 1.4 — Cryptographic Processing**: For registration flows, invokes BCrypt's hash function with the plain-text password and a salt round factor of 10, producing the hashed string to be stored. For login flows, invokes BCrypt's compare function, evaluating the plain-text input against the stored hash without decrypting it.
- **Node 1.5 — Token Generation**: Upon successful authentication, constructs a JWT payload object containing the userId and role fields, signs it using the server's JWT_SECRET environment variable with a specified expiration window, and returns the signed token string to the client.



Level 2 DFD — Tour Management Sub-process Decomposition

- **Node 2.1 — Media Upload Handler**: Receives image files from the admin form submission, constructs an authenticated multipart POST request to the Supabase Storage API, and retrieves the resulting public CDN URLs upon successful upload completion.
- **Node 2.2 — Tour Document Constructor**: Assembles the complete tour MongoDB document object from all submitted form fields — title, description, category, duration, pricePerPerson, itinerary array, and image URL array — and validates it against the Mongoose Tour schema.
- **Node 2.3 — Database Write Executor**: Persists the validated tour document to the tours collection and returns the created document including its auto-generated MongoDB ObjectId to the controller.
- **Node 2.4 — Response Formatter**: Constructs a standardized success response JSON object containing the HTTP 201 status code, a success message string, and the created tour document, dispatching it to the awaiting frontend client.

5.2 Data Dictionary

The Data Dictionary provides a metadata definition for the fields used in the database schema, ensuring consistency across the development lifecycle.

Entity:User

Field Name	Data Type	Description
_id	ObjectId	Unique Primary Key (Auto-generated).
username	String	User's display name.
email	String	Unique email address used for login.
password	String	BCrypt hashed password string.
role	String	Access control tag ('user' or 'admin').
isVerified	Boolean	Flag for email verification status.

Entity:Tour

Field Name	Data Type	Description
title	String	Marketing name of the tour package.
category	String	Enum (Adventure, Wildlife, Heritage, etc.).
pricePerPerson	Number	Base cost per individual.
itinerary	Array[Obj]	Nested array of daily schedules.
images	Array[Str]	List of image URLs (Supabase).
Field Name	Data Type	Description

Entity: Booking

Field Name	Data Type	Description
user	ObjectId	Foreign Key referencing the User collection.
tour	ObjectId	Foreign Key referencing the Tour collection.
preferredStartDate	Date	The user's selected travel date.

guests	Number	Count of travelers in the group.
inquiryStatus	String	Enum: 'pending' (default), 'confirmed', 'rejected'.
finalPrice	Number	Calculated total (pricePerPerson * guests).
depositAmount	Number	40% advance deposit amount calculated from finalPrice
balanceAmount	Number	Remaining 60% balance after deposit deduction
paymentStatus	String (Enum)	Payment lifecycle status: 'pending', 'deposit_paid', 'full_paid', 'refunded'
razorpayDepositOrderId	String	Razorpay order ID for the deposit payment transaction
razorpayDepositPaymentId	String	Razorpay payment ID returned after successful deposit payment
razorpayBalanceOrderId	String	Razorpay order ID for the balance payment transaction
razorpayBalancePaymentId	String	Razorpay payment ID returned after successful balance payment
cancellationRefundAmount	Number	Calculated refund amount upon booking cancellation or rejection

5.3 Process Specification / Activity Flow Diagram

Diagram 1: Core System (Authentication & Tour Management)

This diagram covers how users enter the system and how tours are managed and browsed.

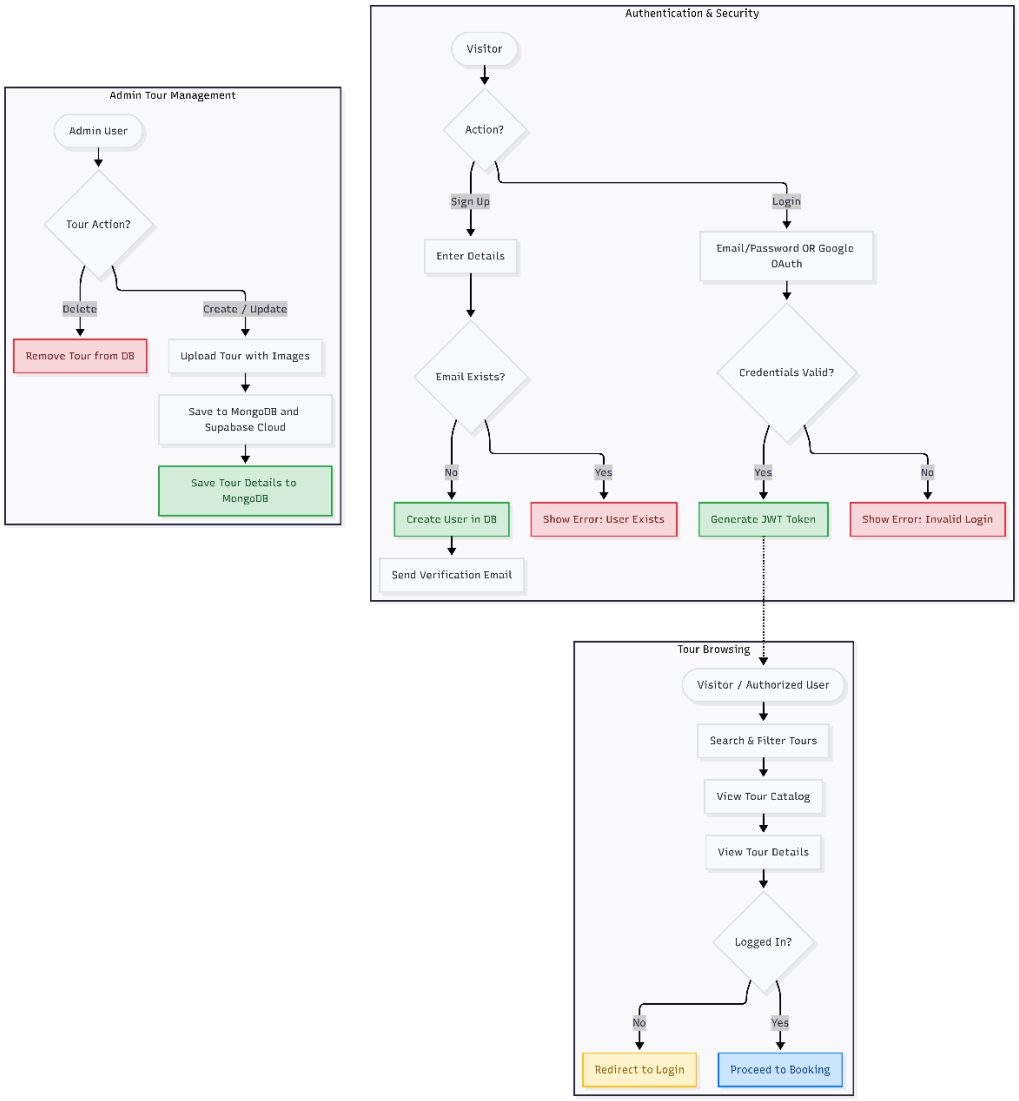
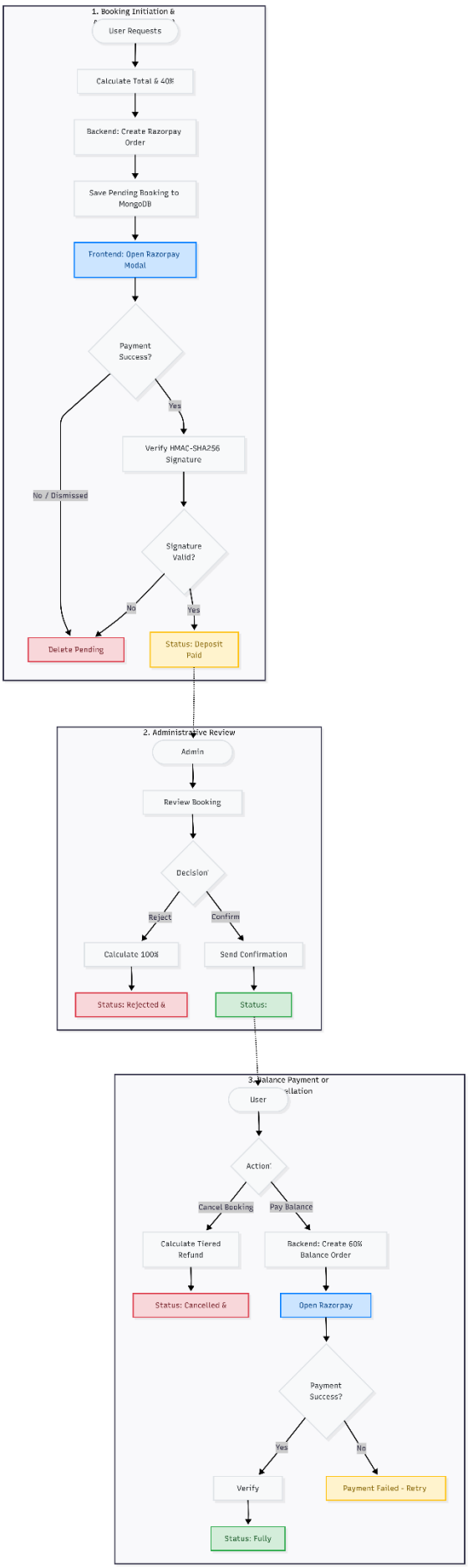
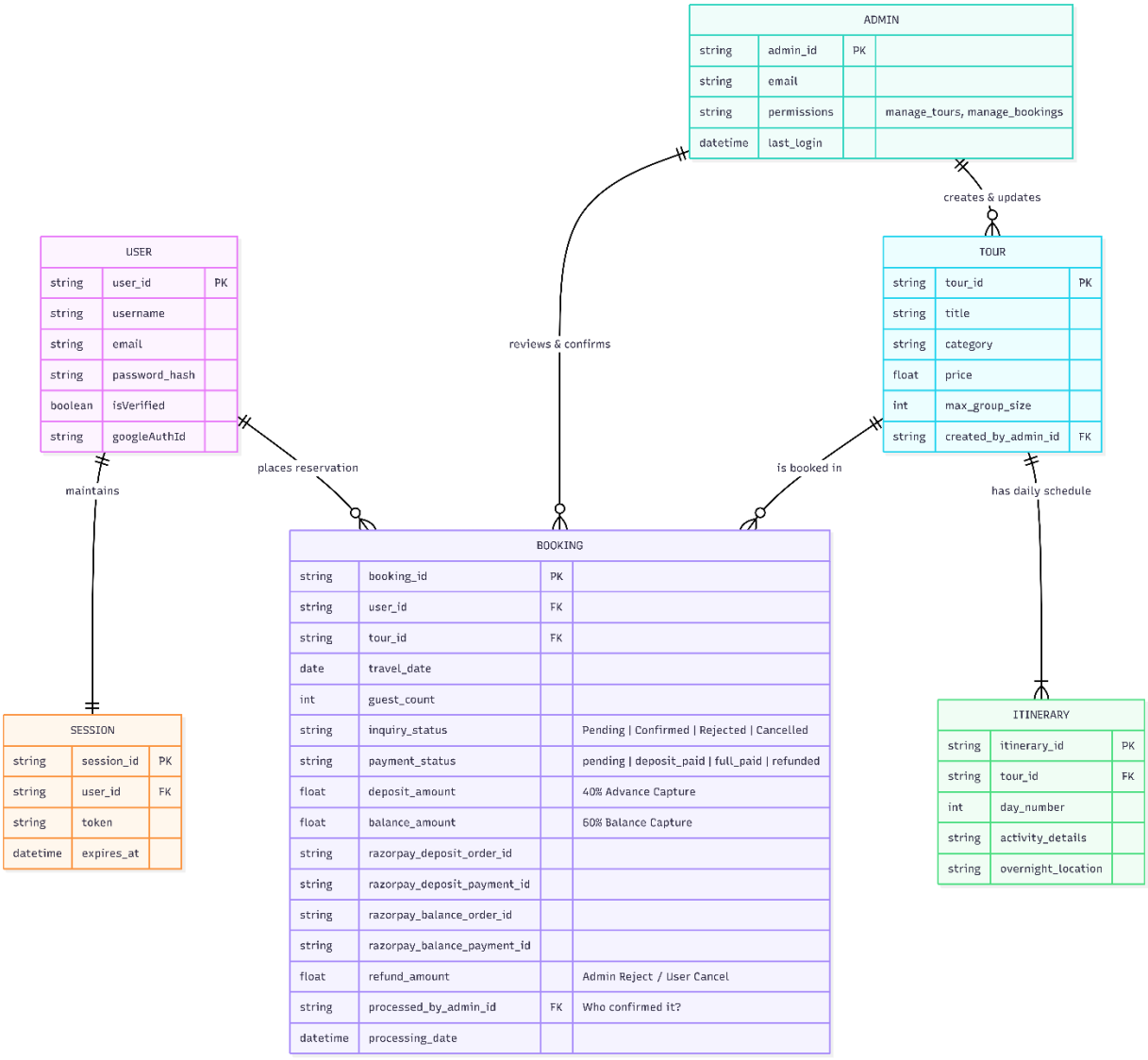


Diagram 2: Complete Booking & Payment Lifecycle

This diagram handles the entire transaction, from the 40% deposit, to the admin review, to the 60% balance payment and cancellation refunds.


1. Booking Initiation &
User Requests
Calculate Total & 40%
Backend: Create Razorpay Order
Save Pending Booking to MongoDB
Frontend: Open Razorpay Modal
Payment Success?
Yes
Verify HMAC-SHA256 Signature
No / Dismissed
Signature Valid?
No
Yes
Delete Pending
Status: Deposit Paid
2. Administrative Review
Admin
Review Booking
Decision
Reject
Confirm
Calculate 100%
Send Confirmation
Status: Rejected &
Status:
3. Balance Payment or
User
Action
Cancel Booking
Pay Balance
Calculate Tiered Refund
Backend: Create 60% Balance Order
Status: Cancelled &
Open Razorpay
Payment Success?
Yes
No
Verify
Payment Failed - Retry
Status: Fully

5.4 Entity-Relationship Diagram / Class Diagram



- Users have a One-to-Many relationship with Bookings (One user can have multiple bookings).
- Tours have a One-to-Many relationship with Bookings (One tour can be booked by many users).
- Users have a One-to-Many relationship with Reviews (Future scope: One user writes many reviews).

Use Case Description

The GlobeTrails system involves two distinct primary actors whose interactions define the complete set of use cases within the system boundary:

Actor 1 — Guest User (Unauthenticated): A visitor who has not logged in or registered. This actor has read-only access to public-facing content.

- UC-G01: Browse Tour Catalog
- UC-G02: View Tour Details Page
- UC-G03: Filter Tours by Category, Price, or Duration
- UC-G04: Register a New Account (Email/Password)
- UC-G05: Register via Google OAuth
- UC-G06: Log In with Email and Password
- UC-G07: Initiate Password Recovery via Email OTP

Actor 2 — Authenticated User (Logged In): A registered user with a valid session token. Inherits all Guest User capabilities plus the following:

- UC-U01: Submit Booking Inquiry for a Tour
- UC-U02: View My Bookings with Status Tracking
- UC-U03: Log Out and Invalidate Client Session

Actor 3 — Administrator (Admin Role): A user with the admin role assigned. Accesses the Admin Panel exclusively. Includes all Authenticated User capabilities plus:

- UC-A01: View Administrative Dashboard with Aggregate Statistics
- UC-A02: Create New Tour Package with Itinerary and Images
- UC-A03: Edit Existing Tour Package Details
- UC-A04: Delete Tour Package from Public Catalog
- UC-A05: View All Booking Inquiries Across All Users
- UC-A06: Confirm or Reject Booking Inquiry
- UC-A07: View All Registered Users
- UC-A08: Delete User Account

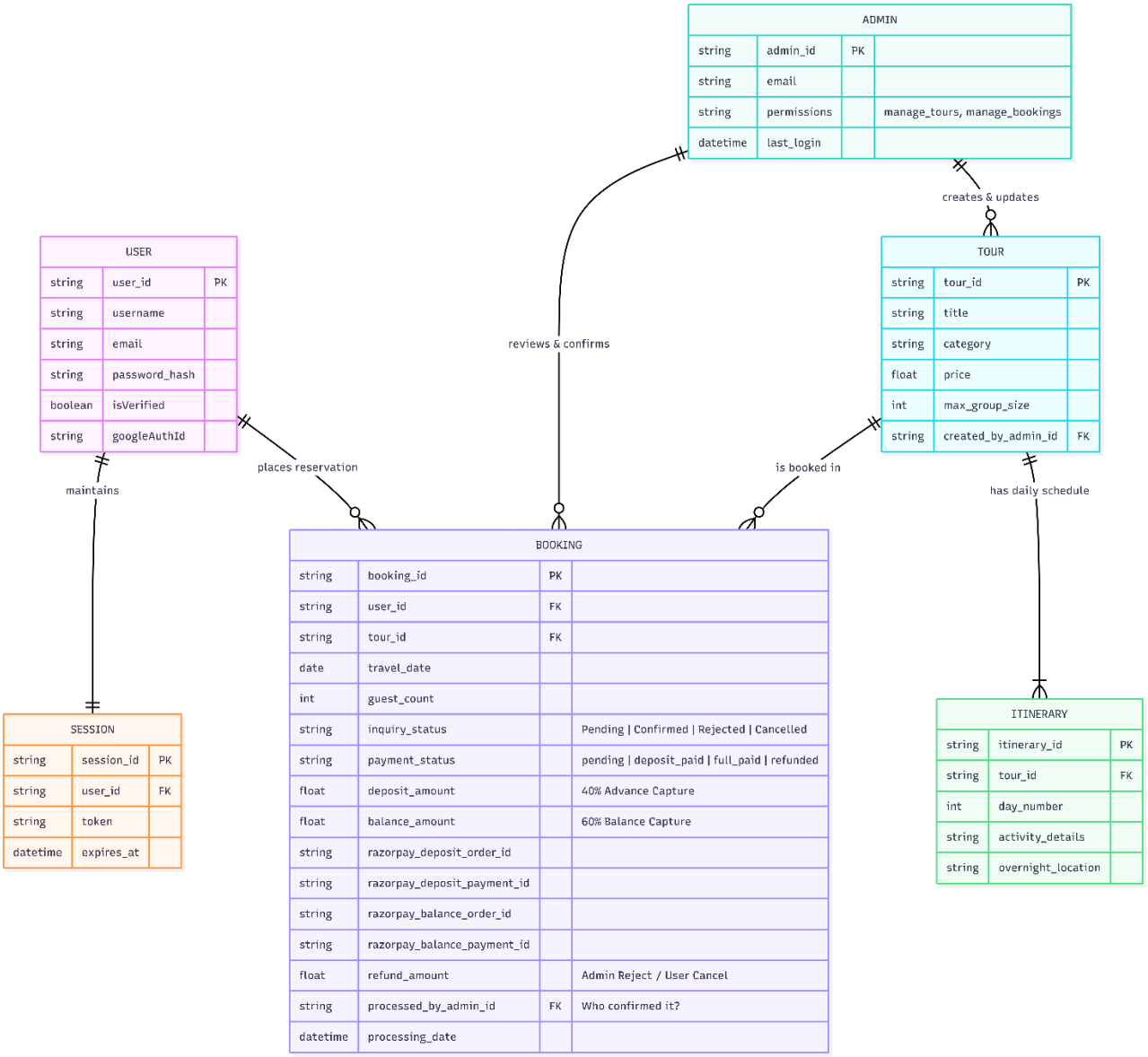
6. System Design

6.1 Database Design

The database is designed using MongoDB's document-oriented model. Normalization: Data is semi-normalized. While Users and Tours are separate collections, Bookings reference them via ObjectId. However, Tour documents allow nested arrays (itinerary, images) for faster read performance (denormalization for read efficiency).

Key Indexing:

- email in User collection (Unique Index).
- category in Tour collection (for faster filtering).



6.2 Input Design

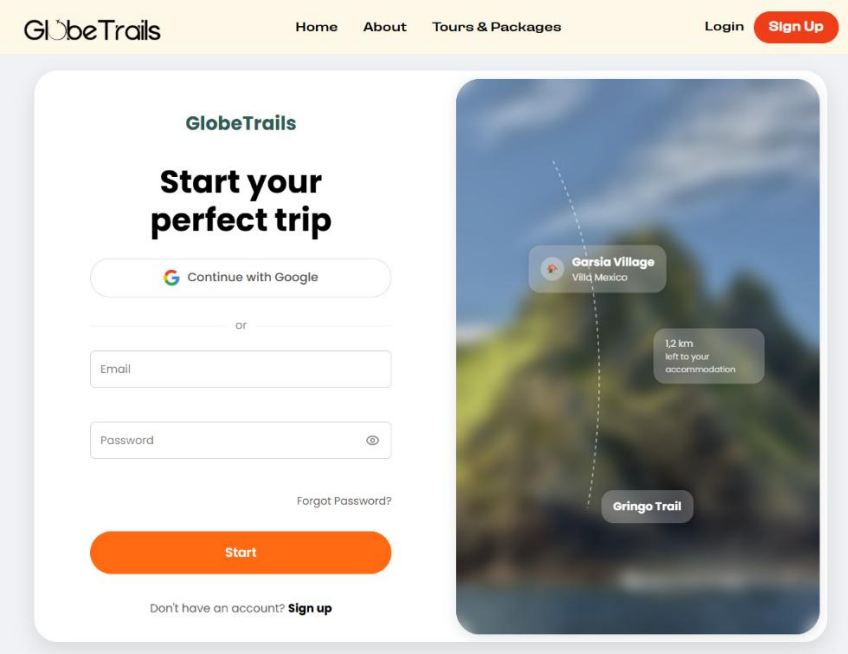
- Forms:

Login Form: Email/Password fields with regex validation.

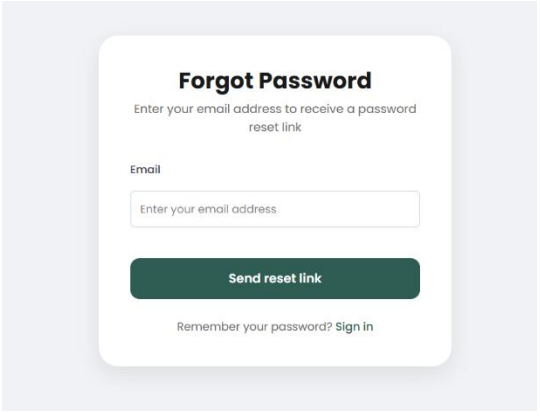
Booking Form: Date picker (min date = today), Guest counter (numeric only).

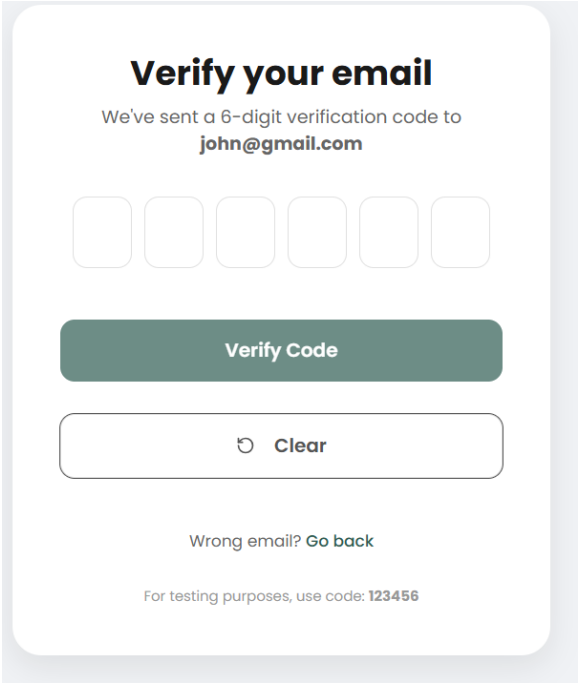
Tour Upload Form: Dynamic inputs for adding multiple itinerary days on the fly.

➢ Login Form:

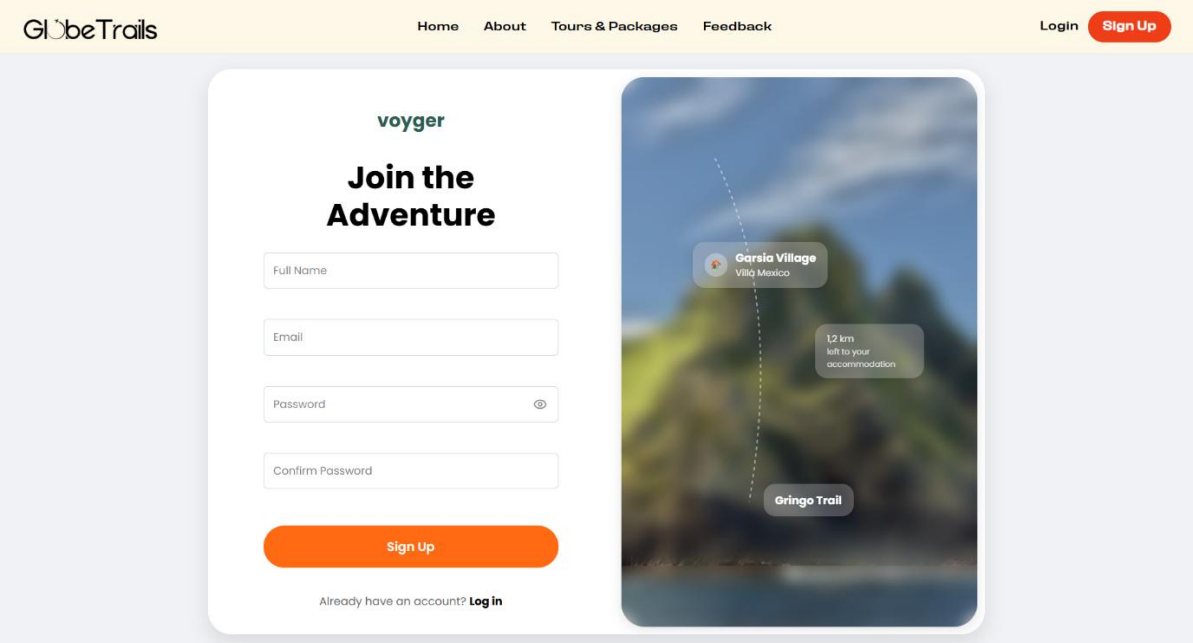


➢ Forget Password & Verify Email :

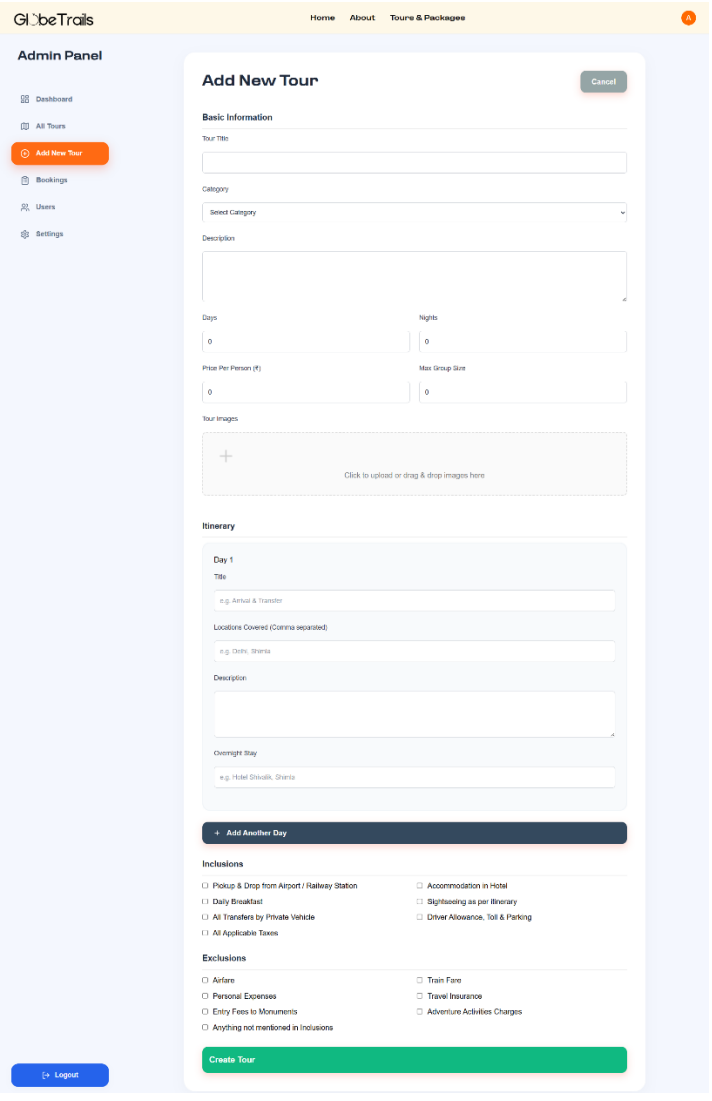




- Register Form:



Booking Form:



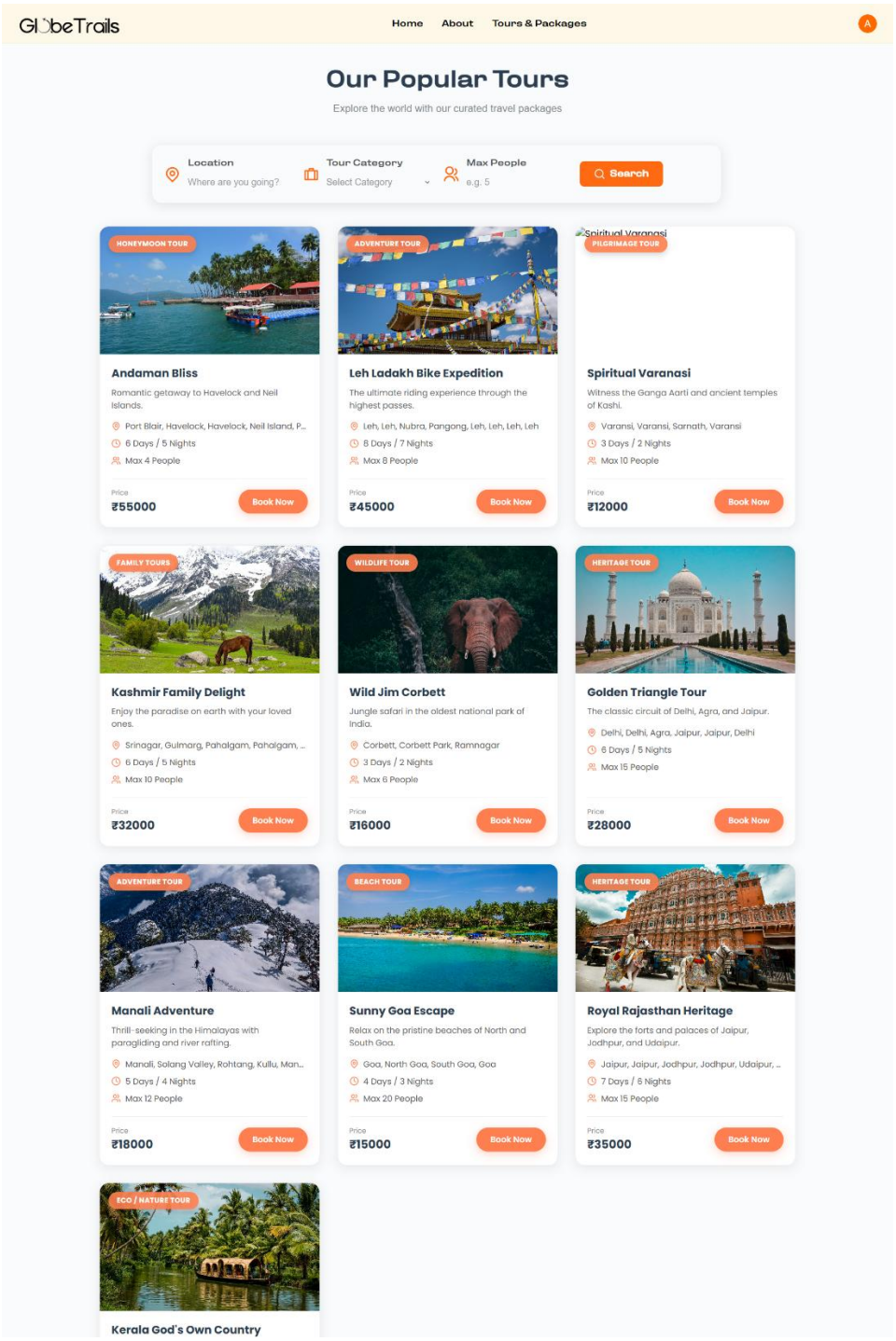
- Security Considerations:

Input Sanitization: All API inputs are validated using mongoose schema validation.

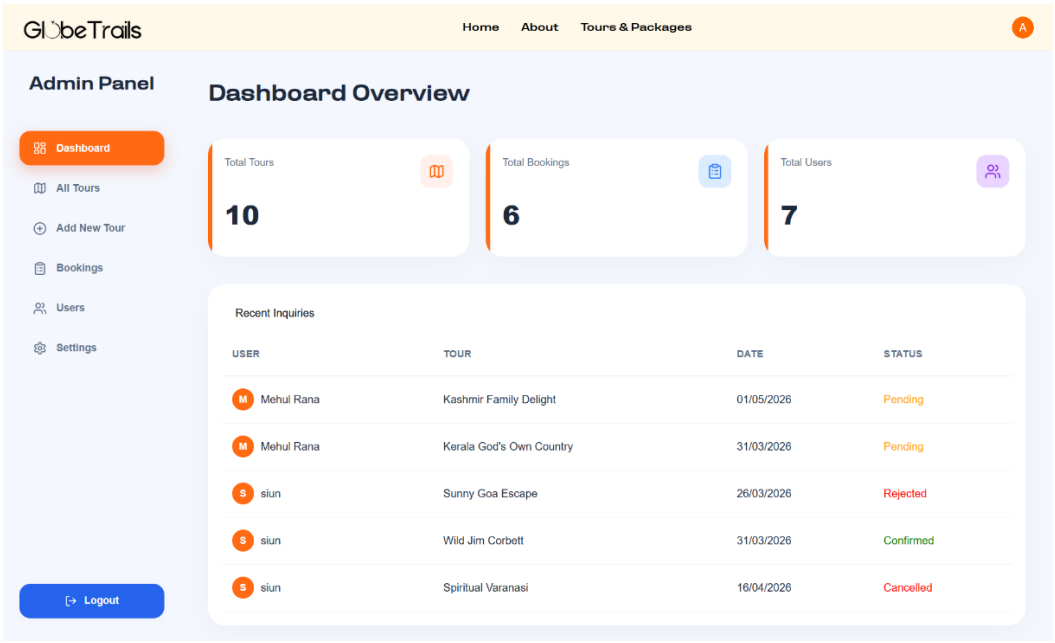
CSRF Protection: Handled via standard header checking and CORS policies.

6.3 Output Design

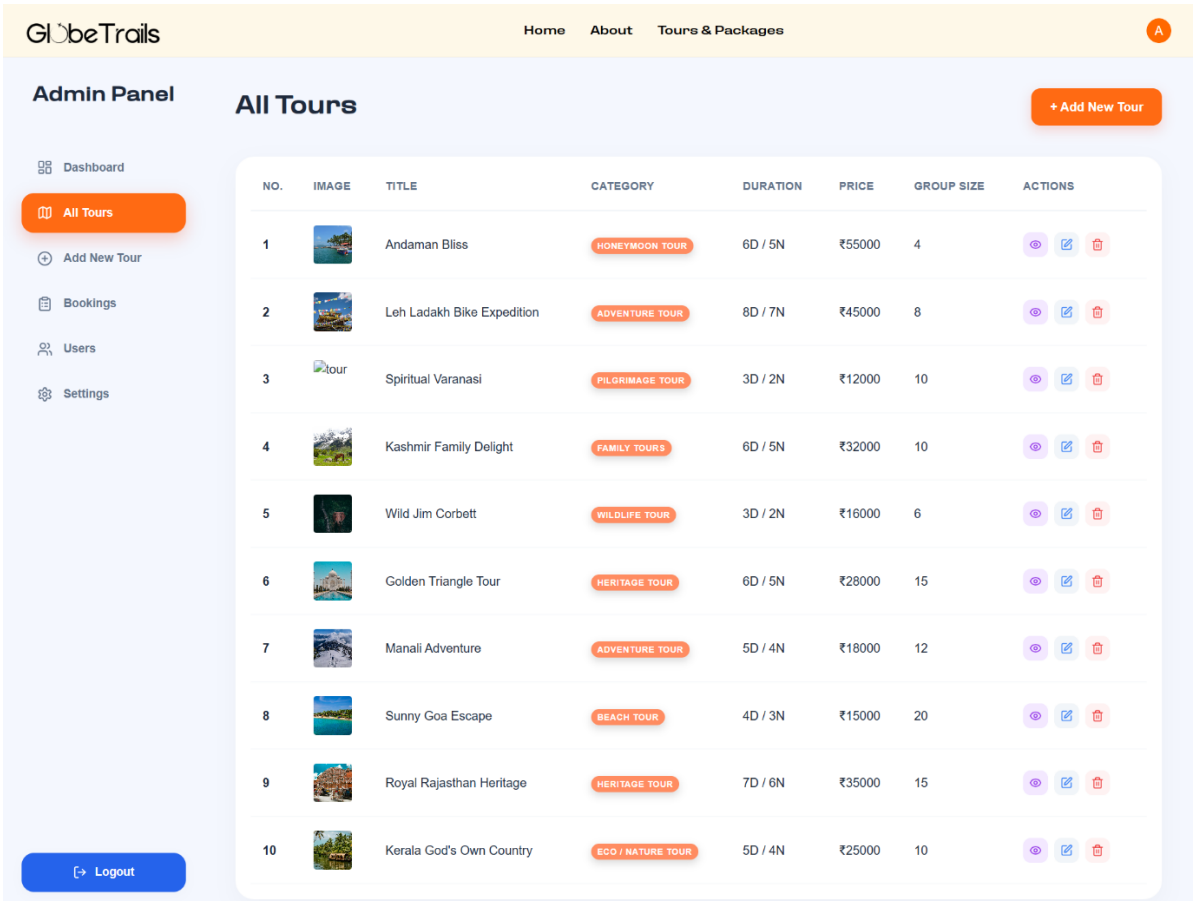
- Tour Cards: Visual cards displaying Image, Title, Price, and Rating.



- Admin Dashboard: Cards showing aggregate statistics (Total Revenue, Active Users).



- Tables: Data grids for listing Users/Bookings with pagination and action buttons (Edit/Delete).



6.4 API Design and Endpoint Specifications

The GlobeTrails backend exposes a comprehensive RESTful API organized into four logical route groups. All protected endpoints require a valid JWT Bearer token in the Authorization request header. The following tables present the complete endpoint specifications:

Authentication Routes (/api/auth)

Method	Endpoint	Access	Description
POST	/api/auth/register	Public	Creates new user account with hashed password
POST	/api/auth/login	Public	Validates credentials, returns signed JWT
GET	/api/auth/google	Public	Initiates Google OAuth 2.0 authorization flow
GET	/api/auth/google/callback	Public	Handles OAuth callback, issues JWT on success
POST	/api/auth/forgot-password	Public	Generates OTP and sends password reset email
POST	/api/auth/verify-otp	Public	Validates OTP, allows password reset

Tour Routes (/api/tours)

Method	Endpoint	Access	Description
GET	/api/tours	Public	Returns all tour documents from the tours collection
GET	/api/tours/:id	Public	Returns a single tour document by ObjectId
POST	/api/tours	Admin Only	Creates a new tour document in the database
PUT	/api/tours/:id	Admin Only	Updates all fields of an existing tour document
DELETE	/api/tours/:id	Admin Only	Permanently removes a tour document

Booking Routes (/api/bookings)

Method	Endpoint	Access	Description
POST	/api/bookings	Authenticated	Creates a new booking inquiry document
GET	/api/bookings/my-bookings	Authenticated	Returns all bookings for the requesting user
GET	/api/bookings	Admin Only	Returns all bookings across all users
PATCH	/api/bookings/:id/status	Admin Only	Updates the inquiryStatus of a specific booking
POST	/bookings	JWT Required	Creates booking + Razorpay deposit order. Returns razorpayOrder object with order ID and amount in paise
POST	/bookings/verify-payment	JWT Required	Verifies Razorpay deposit payment signature using HMAC-SHA256. Updates paymentStatus to 'deposit_paid'
POST	/bookings/create-full-payment	JWT Required	Creates Razorpay order for remaining 60% balance payment
POST	/bookings/verify-full-payment	JWT Required	Verifies Razorpay balance payment signature. Updates paymentStatus to 'full_paid'
DELETE	/bookings/cancel-payment/:id	JWT Required	Permanently deletes a pending (unpaid) booking. Used for cleanup after payment failures

User Management Routes (/api/users)

Method	Endpoint	Access	Description
GET	/api/users	Admin Only	Returns all registered user accounts
DELETE	/api/users/:id	Admin Only	Permanently deletes a user account

7. Software Testing

Software testing was performed to ensure that the GlobeTrails platform is reliable, secure, and functions as intended across different user roles.

7.1 Unit Testing

Unit testing focuses on individual components and backend controllers to verify they work in isolation.

Test_Case ID	Description	Input Data	Expected Output
TC_AUTH_01	User Registration	Username, Email, Password	New User created in DB with isVerified: false
TC_AUTH_02	OTP Verification	6-digit valid OTP	isVerified set to true, Success message shown
TC_AUTH_03	Valid User Login	Correct Email & Password	JWT Token generated and redirected to Home
TC_AUTH_04	Invalid User Login	Incorrect Password	"Incorrect Password" error message displayed
TC_PW_01	Forgot Password	Registered Email	System generates OTP and sends recovery email
Razorpay Order Creation	finalPrice: 18010, guests: 1	Razorpay order created with amount: 720400 paise (40% of 18010 × 100)	Pass
Payment Signature Verification	Valid razorpay_signature	Booking paymentStatus updated to 'deposit_paid', HTTP 200 returned	Pass
Invalid Payment Signature	Tampered razorpay_signature	HTTP 400 returned with "Invalid signature" message, booking unchanged	Pass
Payment Modal Dismiss	User closes Razorpay modal	Pending booking permanently deleted from database	Pass
Payment Failure Cleanup	Razorpay payment.failed event	Pending booking deleted, error toast displayed to user	Pass

7.2 Integration Testing

Integration testing verifies the data flow between the Frontend, Node.js Backend, MongoDB, and Supabase Storage.

Test_Case ID	Description	Input Data	Expected Output
TC_TOUR_01	Admin Uploads Tour	Tour Details + Image Files	Images stored in Supabase; Tour URL saved in MongoDB
TC_BOOK_01	Create Booking Inquiry	Tour ID, Date, Guests	Booking record created with inquiryStatus: "pending"
TC_DASH_01	Dashboard Statistics	Fetch Data from API	Real-time counts of Users, Tours, and Bookings displayed
TC_EMAIL_01	Inquiry Notification	Submit Booking Form	NodeMailer sends "Inquiry Received" email to User
TC_ADM_01	Update Booking Status	Admin clicks "Confirm"	Status updates to "Confirmed" in DB; UI reflects change
End-to-End Deposit Payment	Frontend → POST /bookings → Razorpay Checkout → verify-payment	Booking created with deposit_paid status, Razorpay IDs stored	Pass
Admin Reject with Refund	Admin rejects deposit_paid booking	Booking status: rejected, paymentStatus: refunded, refundAmount recorded	Pass
Balance Payment After Confirmation	Admin confirms → User clicks Pay Balance → Razorpay → verify-full-payment	paymentStatus updated to full_paid	Pass

7.3 System Testing

System testing covers end-to-end user journeys and security constraints of the entire application.

Test_Case ID	Description	Input Data	Expected Output
TC_SYS_01	Full Booking Journey	Search -> Select -> Book	Inquiry successfully appears in Admin Recent Inquiries
TC_SYS_02	Tour Filtering	Click "Adventure Tour" category	Search results display only Adventure-themed tours
TC_SEC_01	Unauthorized Access	Visit /adminpanel as Guest	System redirects visitor to /signin page
TC_SEC_02	Role-Based Control	User tries to access /tour-upload	Access denied; Admin-only route protected by middleware
TC_RESP_01	Responsive UI Test	View site on Mobile (360px)	Navbar converts to hamburger menu; Grid stacks vertically

\

8. Conclusions & Future Enhancements

The GlobeTrails system successfully digitizes the tour booking workflow. It moves beyond a unified brochure to an interactive application.

Future Enhancements:

AI-Driven Tour Recommendation Engine Implementing a collaborative filtering recommendation algorithm would allow the platform to suggest tours to users based on their browsing history, previous bookings, and the behavioral patterns of users with similar profiles. The recommendation logic could be implemented as a separate microservice using Python's scikit-learn library and exposed to the main Node.js backend through a REST API, enabling independent scaling of the recommendation computation without impacting the core application server's performance.

Progressive Web Application (PWA) Conversion Converting GlobeTrails into a Progressive Web App through the addition of a Service Worker script would enable offline functionality for the tour catalog — users who have previously loaded the catalog could browse cached tour information without an active internet connection. The Service Worker would cache static assets and previously fetched tour data using the Cache Storage API, with a network-first update strategy ensuring users always see fresh data when connectivity is available.

Review and Rating System Adding a post-trip review system would allow users who have completed confirmed bookings to submit star ratings and written reviews for specific tours. These reviews would be displayed on the TourDetails page, providing social proof and trust signals that significantly influence the booking decisions of prospective travelers. The review schema would reference both the User and Tour collections, with backend validation ensuring that only users who have a confirmed booking for a specific tour can submit a review for that tour, preventing fraudulent reviews.

Real-Time Seat Availability Tracking The current booking system does not track capacity constraints against available tour seats. Implementing a real-time seat counter using MongoDB's $inc atomic operator for concurrent update safety would allow the system to automatically reject booking requests when a tour's maximum participant capacity has been

reached. This enhancement would also enable the display of "X seats remaining" urgency indicators on the TourDetails page, which is a proven e-commerce conversion optimization technique.

Multi-Language and Multi-Currency Support Expanding the platform's accessibility through internationalization (i18n) would make GlobeTrails accessible to non-English speaking markets. The React i18next library provides a well-established framework for managing translated string resources, while a currency conversion API integration would allow tour prices to be displayed in the visitor's local currency based on their geographic location detected through IP geolocation, significantly reducing the cognitive friction for international visitors evaluating tour pricing.

9. Bibliography & References

1. https://www.w3schools.com/php/php_sessions.asp
2. https://www.w3schools.com/Php/php_mysql_connect.asp
3. https://docs.npmjs.com/
4. https://www.mongodb.com/docs/
5. https://www.mongodb.com/docs/manual/crud/
6. https://react.dev/learn
7. https://nodejs.org/docs/latest/api/
8. https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Serverside/Express_Nodejs/Introduction
9. https://supabase.com/docs/guides/getting-started/quickstarts/reactjs
10. https://razorpay.com/integrations/
11. https://razorpay.com/docs/